ÉTUDE DE CAS — STRIPE

LIVRABLE 4

Modèle de données NoSQL

Certification RNCP 41993 – Adrien COSSON
Bloc 2 - Concevoir et déployer l'infrastructure de données et de calcul pour l'IA

Ce livrable présente le modèle de données NoSQL de l'architecture. Il décrit comment sont stockées les données non structurées et semi-structurées de Stripe (sessions de navigation, logs, avis clients, prédictions du modèle de fraude et features), comment sont gérées les relations entre les documents, quels index sont mis en place et comment ce modèle s'intègre avec les systèmes OLTP et OLAP.

1. Choix des bases NoSQL

Deux bases NoSQL sont utilisées, chacune pour un usage différent :

Base	Type	Rôle
Amazon DocumentDB (compatible MongoDB)	Base orientée documents (JSON)	Stockage des données semi-structurées : sessions, logs, avis clients, prédictions du modèle
Redis (Amazon ElastiCache)	Base clé-valeur en mémoire	Mise à disposition des features ML en quelques millisecondes pour l'inférence

Une base orientée documents est adaptée à ces données car leur structure varie d'un enregistrement à l'autre (un log d'erreur n'a pas les mêmes champs qu'un log d'accès, une session peut contenir 3 ou 50 événements). Il n'est donc pas nécessaire de définir à l'avance des colonnes fixes, comme dans une base relationnelle.

2. Collections DocumentDB

2.1 Collection sessions

Une session correspond à une visite d'un client sur le site ou l'application. Elle contient la liste de ses événements (clickstream) : pages vues, clics, tentatives de paiement.

```
{
  "session_id": "S-4521",
  "customer_id": 42,
  "device": "mobile",
  "ip_country": "FR",
  "started_at": ISODate("2026-10-12T10:02:10Z"),
  "ended_at": ISODate("2026-10-12T10:05:30Z"),
  "events": [
    { "ts": ISODate("2026-10-12T10:02:15Z"), "type": "page_view", "page": "/panier" },
    { "ts": ISODate("2026-10-12T10:02:30Z"), "type": "click", "element": "bouton_payer" },
    { "ts": ISODate("2026-10-12T10:02:41Z"), "type": "payment_attempt",
      "payment_method_id": 881, "transaction_id": 5001 }
  ]
}
```

2.2 Collection logs

Les logs techniques (erreurs, accès) ont une structure commune (date, niveau, service) et un sous-document context dont le contenu varie selon le type de log.

```
{
  "ts": ISODate("2026-10-12T10:02:42Z"),
  "level": "ERROR",
  "service": "api-paiement",
  "type": "error",
  "message": "Timeout lors de l appel au service d inférence",
  "context": { "transaction_id": 5001, "duree_ms": 120, "code_erreur": "INF-504" },
  "session_id": "S-4521",
  "customer_id": 42
}
```

2.3 Collection avis_clients

Les avis et réponses aux enquêtes contiennent du texte libre et des champs facultatifs (pièce jointe, réponses détaillées à une enquête).

```
{
  "review_id": 9001,
  "customer_id": 42,
  "merchant_id": 17,
  "source": "enquete",
  "rating": 2,
  "comment": "Paiement débité deux fois",
  "answers": { "facilite_paiement": 4, "recommanderait": false },
  "attachment_url": "s3://stripe-datalake/avis/9001/capture.png",
  "created_at": ISODate("2026-10-11T18:20:00Z")
}
```

2.4 Collection fraud_predictions

Chaque prédiction du modèle de fraude est enregistrée avec les features utilisées. Cela permet de suivre les performances du modèle et d'expliquer une décision a posteriori (livrable 7).

```
{
  "transaction_id": 5001,
  "customer_id": 42,
  "merchant_id": 17,
  "score": 0.83,
  "decision": "refused",
  "model_version": "fraud-v12",
  "features": {
    "nb_paiements_1h": 6,
    "ratio_montant_moyen": 27.8,
    "nouveau_pays": true,
    "nb_cartes_session": 4
  },
  "predicted_at": ISODate("2026-10-12T10:02:42Z")
}
```

3. Features temps réel dans Redis

Redis stocke uniquement les features déjà calculées, sous forme de clés simples. Chaque clé correspond à un client ou à un marchand et contient un ensemble de valeurs (hash).

```
features:customer:42  ->  { nb_paiements_1h: 6, montant_moyen_30j: 45.0,
                            pays_habituels: "FR", segment: "regulier" }
features:merchant:17  ->  { taux_fraude_30j: 0.012, nb_transactions_1h: 1520 }
```

Le service d'inférence lit ces clés en une seule opération (HGETALL features:customer:42). Une durée de vie (TTL, Time To Live) est appliquée aux clés : les features d'un client inactif depuis plusieurs mois disparaissent automatiquement, ce qui limite la mémoire utilisée.

4. Gestion des données non structurées et semi-structurées

- **Schéma flexible :** les documents d'une même collection partagent des champs communs (identifiants, dates), mais peuvent avoir des champs différents ou facultatifs. Par exemple, le sous-document context des logs varie selon le service qui l'a produit.
- **Données imbriquées :** les événements d'une session sont stockés dans un tableau à l'intérieur du document, ce qui reflète leur structure naturelle et permet de les interroger directement (livrable 8, requête 12).
- **Texte libre :** les commentaires des avis clients sont stockés tels quels et peuvent être recherchés par mots-clés (livrable 8, requête 14).
- **Données XML :** certaines sources externes (rapports bancaires par exemple) envoient du XML. Les informations utiles sont converties en JSON lors de l'ingestion, et le fichier d'origine est conservé dans le Data Lake S3.
- **Données binaires :** les fichiers volumineux (captures d'écran, pièces jointes) ne sont pas stockés dans DocumentDB. Ils sont déposés dans S3 et le document ne contient que leur lien (attachment_url). On évite ainsi d'alourdir la base, dont les documents sont limités à 16 Mo.

- **Contrôle minimum :** avant l'écriture, le connecteur d'ingestion vérifie que les champs obligatoires (identifiants, date) sont présents, afin d'éviter des documents inutilisables.

5. Gestion des relations entre documents

Pour chaque relation, deux stratégies sont possibles : imbriquer les données dans le même document (embedding), ou ne stocker que l'identifiant de l'autre document (référencement). Le choix dépend de la façon dont les données sont lues.

Relation	Stratégie	Justification
Session → événements	Imbrication	Les événements sont toujours lus avec leur session et leur nombre est limité à une visite
Prédiction → features utilisées	Imbrication	Les features sont propres à cette prédiction et ne changent plus après coup
Avis → réponses à l'enquête	Imbrication	Les réponses n'ont de sens qu'avec l'avis auquel elles appartiennent
Session, avis, prédiction → client	Référencement (customer_id)	Le client est partagé par de nombreux documents ; ses informations restent dans l'OLTP
Avis, prédiction → marchand	Référencement (merchant_id)	Même raison : une seule source pour les informations du marchand
Prédiction, log → transaction	Référencement (transaction_id)	La transaction est enregistrée dans l'OLTP, qui reste la référence
Log → session	Référencement (session_id)	Permet de relier une erreur au parcours du client

La règle retenue est la suivante : on imbrique ce qui est lu ensemble et reste de taille limitée, et on référence ce qui est partagé entre plusieurs documents ou peut grossir sans limite. Les informations détaillées des clients et des marchands ne sont donc jamais recopiées dans DocumentDB, ce qui évite les incohérences et limite la présence de données personnelles.

6. Stratégie d'indexation

Les index sont choisis en fonction des requêtes réellement utilisées (livrables 7 et 8). Chaque index accélère les lectures mais ralentit légèrement les écritures : seuls les champs utiles sont donc indexés.

Collection	Index	Utilité
sessions	{ customer_id: 1, started_at: -1 }	Retrouver les dernières sessions d'un client (calcul des features)
sessions	{ started_at: 1 } avec TTL	Rechercher les sessions récentes (requête 12) et les supprimer automatiquement après 13 mois
logs	{ service: 1, level: 1, ts: -1 }	Retrouver rapidement les erreurs d'un service
logs	{ ts: 1 } avec TTL	Supprimer automatiquement les logs après 90 jours
avis_clients	{ merchant_id: 1, created_at: -1 }	Avis d'un marchand, du plus récent au plus ancien (requête 13)
avis_clients	{ customer_id: 1 }	Retrouver tous les avis d'un client (droit d'accès et d'effacement RGPD)
fraud_predictions	{ transaction_id: 1 } unique	Une seule prédiction par transaction, et accès direct depuis une transaction
fraud_predictions	{ predicted_at: 1, model_version: 1 }	Suivi du modèle par jour et par version (requête 15)
fraud_predictions	{ predicted_at: 1 } avec TTL	Supprimer automatiquement les prédictions après 13 mois

Les index TTL (Time To Live) suppriment automatiquement les documents trop anciens. Ils appliquent ainsi les durées de conservation définies dans le livrable 6 ; les données restent disponibles dans le Data Lake S3 puis dans S3 Glacier pour l'historique.

Si le volume augmente fortement, les collections peuvent être fragmentées (sharding) sur plusieurs machines avec DocumentDB Elastic Clusters, en utilisant customer_id comme clé de répartition. Les documents d'un même client restent alors sur la même machine, ce qui garde les requêtes rapides (livrable 1, section 7).

7. Intégration avec les systèmes OLTP et OLAP

- **Identifiants communs :** les documents utilisent les mêmes identifiants que l'OLTP (customer_id, merchant_id, transaction_id). Une donnée de DocumentDB peut donc toujours être reliée à la transaction, au client ou au marchand correspondant dans Aurora PostgreSQL.
- **Alimentation depuis l'OLTP et les applications :** les données arrivent par Kafka (livrable 5). Les sessions, logs et avis viennent directement du site et des applications. Les prédictions sont publiées dans Kafka par le service d'inférence, puis écrites dans DocumentDB par le même connecteur.
- **Vers l'OLAP :** Kafka copie également ces données dans le Data Lake S3. L'ELT orchestré par Airflow les charge ensuite dans Redshift, par exemple pour analyser les avis par marchand ou les fraudes par appareil. DocumentDB n'est donc pas interrogé directement par les analyses lourdes, ce qui préserve ses performances.
- **Vers le Machine Learning :** les sessions servent au calcul des features stockées dans Redis, et la collection fraud_predictions sert au suivi du modèle (livrable 7).
- **Droits des personnes :** grâce à l'index sur customer_id, toutes les données d'un client peuvent être retrouvées ou supprimées rapidement en cas de demande RGPD.

8. Synthèse

Collection / clé	Contenu	Relations	Principaux index
sessions	Visites et événements de navigation	Événements imbriqués ; customer_id référencé	customer_id + started_at, TTL
logs	Logs techniques (erreurs, accès)	session_id, transaction_id référencés	service + level + ts, TTL
avis_clients	Avis et réponses aux enquêtes	Réponses imbriquées ; client et marchand référencés	merchant_id + created_at, customer_id
fraud_predictions	Prédictions du modèle de fraude	Features imbriquées ; transaction référencée	transaction_id (unique), predicted_at, TTL
Redis features:*	Features temps réel	Clé = identifiant du client ou du marchand	Accès direct par clé, TTL